

OSSP (Operating Systems And Systems Programming) - 25CS2104E

PRACTICAL 5

Objective: Implement producer-consumer communication with an anonymous pipe, measure communication efficiency, and execute the equivalent of `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()` and `exec()`.

Step 1 — Open Ubuntu Terminal

Press `Ctrl + Alt + T`.

`Ctrl + Alt + T`

You should see something similar to:

Ubuntu terminal opens.

Step 2 — Create the Practical Folder

Create and enter the folder.

```
mkdir -p ~/ossp_practical5
cd ~/ossp_practical5
pwd
```

Step 3 — Understand pipe()

An anonymous pipe has two ends: `pipefd[0]` reads and `pipefd[1]` writes.

```
int pipefd[2];
pipe(pipefd);
```

You should see something similar to:

Producer/Parent → `pipefd[1]` → PIPE → `pipefd[0]` → Consumer/Child

Step 4 — Create Producer–Consumer Program

Open the source file.

```
nano producer_consumer.c
```

You should see something similar to:

GNU nano opens.

Paste this program:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>
#include <stdlib.h>
```

```
int main() {
    int pipefd[2];
    pid_t pid;
    char buffer[100];

    if (pipe(pipefd) == -1) {
```

```

    perror("pipe");
    return 1;
}

pid = fork();

if (pid < 0) {
    perror("fork");
    return 1;
}

if (pid == 0) {
    close(pipefd[1]);
    read(pipefd[0], buffer, sizeof(buffer));
    printf("Consumer received: %s
", buffer);
    close(pipefd[0]);
    exit(0);
}

close(pipefd[0]);
strcpy(buffer, "Hello from Producer");
printf("Producer sending: %s
", buffer);
write(pipefd[1], buffer, strlen(buffer) + 1);
close(pipefd[1]);

wait(NULL);
printf("Communication completed
");
return 0;
}

```

Step 5 — Compile Producer–Consumer

Compile the program.

```
gcc producer_consumer.c -o producer_consumer
```

Step 6 — Run Producer–Consumer

Run the program and observe the data transfer.

```
./producer_consumer
```

Step 7 — Understand the Communication Flow

The parent writes the message and the child reads it.

You should see something similar to:

```

Parent / Producer
|
write()
|
PIPE
|
read()
|
Child / Consumer

```

Step 8 — Measure Communication Efficiency

For a larger experiment, record start and end time using `clock_gettime(CLOCK_MONOTONIC, ...)`.

Record: number of messages, total elapsed time, and approximate throughput.

You should see something similar to:

Example observation:

Messages sent: 10000

Elapsed time: 0.012 seconds

Throughput: approximately 833333 messages/second

These are example values. Your measured result depends on your system and message size.

Step 9 — Create the Pipeline Program

The second part must reproduce `ls -l | grep ".c"` using system calls.

`nano pipeline.c`

You should see something similar to:

GNU nano opens.

Paste this program:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int pipefd[2];
    pid_t p1, p2;

    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    p1 = fork();

    if (p1 < 0) {
        perror("fork");
        return 1;
    }

    if (p1 == 0) {
        close(pipefd[0]);
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);
        perror("exec ls");
        exit(1);
    }

    p2 = fork();

    if (p2 < 0) {
```

```

    perror("fork");
    return 1;
}

if (p2 == 0) {
    close(pipefd[1]);
    dup2(pipefd[0], STDIN_FILENO);
    close(pipefd[0]);

    execlp("grep", "grep", ".c", NULL);
    perror("exec grep");
    exit(1);
}

close(pipefd[0]);
close(pipefd[1]);

waitpid(p1, NULL, 0);
waitpid(p2, NULL, 0);

return 0;
}

```

Step 10 — Compile the Pipeline Program

Compile with GCC.

```
gcc pipeline.c -o pipeline
```

Step 11 — Run the Pipeline

Run the program.

```
./pipeline
```

Your filenames, sizes, owners and dates will differ. Only lines containing `.c` should pass through `grep`.

Step 12 — Understand `dup2()`

Connect the pipe to standard output of `ls` and standard input of `grep`.

```

dup2(pipefd[1], STDOUT_FILENO);
dup2(pipefd[0], STDIN_FILENO);

```

You should see something similar to:

```
ls -l → pipe → grep ".c" → terminal
```

Step 13 — Understand `exec()`

`exec` replaces each child process image with the required command.

```

execlp("ls", "ls", "-l", NULL);
execlp("grep", "grep", ".c", NULL);

```

You should see something similar to:

Child 1 becomes `ls`.
 Child 2 becomes `grep`.
 The parent waits for both children.

Step 14 — Compare with the Shell Command

The C program should behave like this shell pipeline.

```
ls -l | grep ".c"
```

Step 15 — Trace the System Calls

Use strace to inspect pipe, process creation, dup2 and exec operations.

```
strace -f ./pipeline 2> pipeline_trace.txt  
cat pipeline_trace.txt | grep -E 'pipe|clone|fork|dup2|execve'
```

Exact syscall names and paths can vary slightly by Linux distribution.

Step 16 — Final Test

Run both programs and record the final output.

```
./producer_consumer  
./pipeline
```

FINAL RESULT: Anonymous-pipe producer/consumer communication works, communication efficiency can be measured, and `ls -l | grep ".c"` is reproduced using `fork()`, `pipe()`, `dup2()` and `exec()`.